

UNIVERSIDAD PRIVADA DOMINGO SAVIO  
FACULTAD DE INGENIERÍA — PROGRAMACIÓN IV

GUÍA PRÁCTICA COMPLETA  
StudySync en Tiempo Récord  
Sistema Distribuido: APIs REST · Redis Pub/Sub · JWT · Supabase · Swagger · Producción

|              |                                                                                     |
|--------------|-------------------------------------------------------------------------------------|
| Para quién   | Programación IV, 7.º semestre, Ing. de Sistemas                                     |
| Objetivo     | Implementar StudySync completo desde cero en 10 días hábiles                        |
| Nivel        | Sin experiencia previa en APIs ni bases de datos en la nube — todo desde cero       |
| Lenguaje     | Node.js + Express (ejemplos principales) · Python/FastAPI (notas comparativas)      |
| Tiempo total | ~56 horas · 18 Mayo → 3 Junio 2026 · 10 días hábiles                                |
| Costo        | <b>\$0 — Todas las herramientas son gratuitas y no requieren tarjeta de crédito</b> |
| Autor        | M.Sc. Jimmy Nataniel Requena Llorentty · UPDS 2026                                  |

## Antes de Empezar — Lee Esto

### ¿Qué vamos a construir?

🔗 StudySync es una plataforma para coordinar grupos de estudio universitarios. Imagínala como un WhatsApp de sesiones de estudio, pero construido por ti: cuando alguien crea una sesión, todos los interesados la ven en tiempo real sin recargar la página. Tiene usuarios con contraseña segura, una base de datos real en la nube, y está publicado en internet para que cualquier persona del mundo pueda usarlo.

| Bloque    | Tecnología                       | Para qué sirve                      | Paso en esta guía |
|-----------|----------------------------------|-------------------------------------|-------------------|
| Setup     | Node.js + VS Code + GitHub       | El taller donde se construye todo   | Pasos 1-2         |
| Bloque 1  | Express.js (API REST)            | El mesero que recibe pedidos        | Pasos 3-5         |
| Bloque 2  | Redis Pub/Sub + Socket.io        | El sistema de radio interna         | Pasos 6-8         |
| Bloque 3  | JWT + bcrypt + Supabase + Prisma | La puerta con llave y la bodega     | Pasos 9-12        |
| Bloque 3b | Swagger + CORS + Rate Limiting   | El menú y los guardias de seguridad | Pasos 13-14       |
| Bloque 4  | CloudFormation + LocalStack      | Los planos de la infraestructura    | Paso 15           |
| Cierre    | Render + QA + Demo               | Abrir el restaurante al público     | Pasos 16-18       |

### Reglas de oro para avanzar rápido

- ▶ Si una tarea tarda más de 2 horas: hacer commit de lo que hay, saltar a la siguiente, volver con mente fresca.
- ▶ El solucionario de errores está en cada paso. Si aparece un error, busca su texto exacto en la sección '🐛 Si algo falla'.
- ▶ Nunca copies código sin leer la explicación de arriba. La explicación es lo que te permite defenderlo en la sustentación.
- ▶ Cada bloque de código tiene un RESULTADO ESPERADO en verde. Si no ves ese resultado, hay un error que resolver antes de continuar.

⚠️ **Instala Todo en el Paso 1 antes de escribir una sola línea de código. El 60% de los errores de los primeros días vienen de instalaciones incompletas o desactualizadas.**

## PASO 1 Instalar todas las herramientas

Tiempo estimado: 45 minutos · Solo se hace una vez

### 1.1 Node.js 20 LTS

🕒 Node.js es el motor que hace correr tu código JavaScript fuera del navegador. Piensa en él como el cocinero que puede trabajar en la cocina aunque no haya ningún cliente en el restaurante.

Descarga Node.js desde el sitio oficial. La versión LTS (Long Term Support) es la más estable.

🖱️ Windows: Abrir el navegador → ir a [nodejs.org](https://nodejs.org) → hacer clic en el botón verde grande que dice '20.x.x LTS' → descargar el instalador .msi → ejecutarlo → clic en Next, Next, Next, Install, Finish. El instalador configura el PATH automáticamente.

#### ▶ VERIFICAR INSTALACIÓN

```
# Abrir PowerShell o CMD y verificar la instalación:  
node --version  
npm --version
```

#### ▶ RESULTADO ESPERADO

```
v20.x.x  
10.x.x
```

💡 Si el comando 'node' no se reconoce después de instalar, cerrar y volver a abrir PowerShell. El PATH se actualiza al reiniciar la terminal.

### 1.2 Git para Windows

🕒 Git es como el historial de guardado de un videojuego. Cada vez que haces commit, guardas un punto al que puedes volver si algo sale mal. GitHub es donde guardas esas copias en la nube.

🖱️ Windows: Ir a [git-scm.com](https://git-scm.com) → clic en 'Download for Windows' → ejecutar el instalador → dejar todas las opciones por defecto → clic en Next hasta Finish. El instalador agrega Git al PATH automáticamente.

#### ▶ VERIFICAR GIT

```
git --version
```

#### ▶ RESULTADO ESPERADO

```
git version 2.x.x
```

### 1.3 VS Code

VS Code es el editor de código. Es gratuito, liviano y tiene todas las extensiones que necesitas.

🖱️ Windows: Ir a [code.visualstudio.com](https://code.visualstudio.com) → clic en 'Download for Windows' → ejecutar el instalador → marcar la opción 'Add to PATH' si aparece → Next, Install, Finish.

Extensiones esenciales a instalar después de abrir VS Code (Ctrl+Shift+X):

- ▶ Thunder Client — para probar los endpoints de la API (reemplaza Postman)
- ▶ YAML (Red Hat) — detecta errores en archivos .yaml en tiempo real
- ▶ ESLint — resalta errores de JavaScript mientras escribes
- ▶ GitLens — muestra quién cambió cada línea de código

▶ **VERIFICAR VS CODE**

```
code --version
```

▶ **RESULTADO ESPERADO**

```
1.xx.x
```

## 1.4 Docker Desktop (para LocalStack en el Paso 15)

 **Windows:** Ir a [docker.com/products/docker-desktop](https://docker.com/products/docker-desktop) → 'Download for Windows' → ejecutar instalador → reiniciar Windows cuando lo pida. Abrir Docker Desktop y esperar que el ícono de la ballena aparezca en la barra de tareas.

▶ **VERIFICAR DOCKER**

```
docker --version
```

▶ **RESULTADO ESPERADO**

```
Docker version 24.x.x, build xxxxxxxx
```

 **Si Docker no arranca en Windows, puede necesitar activar WSL2. Seguir las instrucciones en: [docs.docker.com/desktop/install/windows-install](https://docs.docker.com/desktop/install/windows-install) — sección 'Prerequisites'.**

## 1.5 Crear las cuentas gratuitas

Estas cuentas se crean UNA sola vez y se usan durante todo el curso. Ninguna requiere tarjeta de crédito.

| Servicio        | URL                                             | Plan gratuito            | Para qué                   |
|-----------------|-------------------------------------------------|--------------------------|----------------------------|
| <b>GitHub</b>   | <a href="https://github.com">github.com</a>     | Repos ilimitados         | Código fuente + despliegue |
| <b>Upstash</b>  | <a href="https://upstash.com">upstash.com</a>   | 10.000 cmd/día · 256 MB  | Redis Pub/Sub gratuito     |
| <b>Supabase</b> | <a href="https://supabase.com">supabase.com</a> | 500 MB · 2 proyectos     | PostgreSQL en la nube      |
| <b>Render</b>   | <a href="https://render.com">render.com</a>     | 750 h/mes · SSL incluido | Despliegue de la API       |

 **Crear todas las cuentas con el mismo email universitario y usar 'Login with GitHub' donde esté disponible. Eso conecta todo en un solo lugar y elimina el problema de recordar múltiples contraseñas.**

## PASO 2 Crear la estructura del proyecto

Tiempo estimado: 20 minutos · La base de todo lo demás

### 2.1 Crear la carpeta y el repositorio

 Abrir PowerShell → navegar a la carpeta de documentos → ejecutar los comandos uno por uno:

#### ► CREAR PROYECTO

```
# 1. Ir a tu carpeta de proyectos (ajusta la ruta a tu usuario)
cd C:\Users\TuNombre\Documents

# 2. Crear la carpeta del proyecto
mkdir studysync-api
cd studysync-api

# 3. Inicializar Node.js (crea el package.json)
npm init -y

# 4. Inicializar Git
git init
```

#### ► RESULTADO ESPERADO

```
Wrote to ...package.json
Initialized empty Git repository in .../studysync-api/.git/
```

### 2.2 Instalar dependencias base

🕒 Las dependencias son como los ingredientes de una receta. Antes de cocinar, necesitas tener todo en la despensa. npm install descarga esos ingredientes a la carpeta node\_modules.

#### ► INSTALAR DEPENDENCIAS

```
# Instalar las dependencias principales de una vez
npm install express dotenv

# Instalar nodemon como dependencia de desarrollo
# (nodemon reinicia el servidor automáticamente cuando cambias el código)
npm install -D nodemon
```

#### ► RESULTADO ESPERADO

```
added 57 packages, and audited 58 packages in 4s
```

### 2.3 Crear la estructura de carpetas

StudySync sigue el patrón MVC (Modelo-Vista-Controlador). Esto significa que el código está organizado por responsabilidad, no por tecnología. Cada carpeta tiene un solo tipo de trabajo.

#### ► CREAR ESTRUCTURA (Windows CMD)

```
# Crear todas las carpetas de una vez
mkdir src
mkdir src\routes
mkdir src\controllers
mkdir src\middlewares
mkdir src\redis
```

```
mkdir src\subscribers
mkdir src\swagger
mkdir public

# Crear los archivos principales vacíos
type nul > src\server.js
type nul > src\app.js
type nul > src\db.js
type nul > .env
type nul > .gitignore
```

Si prefieres hacerlo en PowerShell:

```
► CREAR ESTRUCTURA (PowerShell)
# PowerShell equivalente
New-Item -ItemType Directory -Path
src,src\routes,src\controllers,src\middlewares,src\redis,src\subscribers,src\swagger,public
-Force
New-Item -ItemType File -Path src\server.js,src\app.js,src\db.js,.env,.gitignore -Force
```

## 2.4 Configurar .gitignore y package.json

El archivo .gitignore le dice a Git qué archivos NUNCA debe subir a GitHub. El más importante es .env porque contiene contraseñas.

Abrir VS Code en la carpeta del proyecto: code . (con el punto, que significa 'esta carpeta')

Abrir el archivo .gitignore y escribir:

```
► CONTENIDO DE .gitignore
node_modules/
.env
*.log
dist/
.DS_Store
```

Abrir el archivo package.json y agregar el bloque 'scripts' (reemplazar el que generó npm init):

```
► package.json COMPLETO
{
  "name": "studysync-api",
  "version": "1.0.0",
  "description": "Sistema de coordinación de grupos de estudio",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js"
  },
  "engines": {
    "node": ">=20.0.0"
  }
}
```

## 2.5 Configurar el archivo .env

● El archivo .env contiene credenciales privadas. NUNCA debe subirse a GitHub. Verificar que .env está en .gitignore ANTES de hacer el primer commit.

Abrir el archivo `.env` y escribir la estructura base (los valores reales se agregan en los pasos siguientes):

```
► ARCHIVO .env BASE
PORT=3000
NODE_ENV=development

# Redis (se completa en el Paso 6)
REDIS_URL=redis://default:PENDIENTE@PENDIENTE:6379

# Base de datos (se completa en el Paso 10)
DATABASE_URL=postgresql://postgres:PENDIENTE@PENDIENTE:5432/postgres

# JWT (se completa en el Paso 9)
JWT_SECRET=studysync_secreto_super_seguro_cambialo_en_produccion_2026
JWT_EXPIRES_IN=1h
```

## 2.6 Primer commit en GitHub

Crear un repositorio en github.com: ir a github.com → hacer clic en '+' (arriba a la derecha) → New repository → nombre: studysync-api → seleccionar Public → NO marcar 'Add a README' → Create repository.

Luego conectar tu proyecto local con GitHub:

```
► PRIMER COMMIT
git add .
git commit -m "feat: setup inicial del proyecto StudySync"
git branch -M main
git remote add origin https://github.com/TUUSUARIO/studysync-api.git
git push -u origin main
```

```
► RESULTADO ESPERADO
Branch 'main' set up to track remote branch 'main' from 'origin'.
```

✓ Al finalizar este paso debes tener: Node.js + Git + VS Code instalados. Proyecto en GitHub. Estructura de carpetas creada. package.json con scripts start y dev. .env en .gitignore.

## PASO 3 El primer servidor Express

Tiempo estimado: 30 minutos · El corazón de la API

🔗 Express es como el edificio del restaurante: sin él, no hay lugar donde los clientes puedan hacer pedidos. `src/server.js` enciende el edificio y `src/app.js` define cómo está organizado por dentro.

### 3.1 Crear `src/app.js` — La configuración de Express

`app.js` es donde se define qué puede hacer la API: qué rutas existen, qué middlewares se aplican, cómo se manejan los errores. Abrir el archivo `src/app.js` y escribir:

```
▶ src/app.js
// src/app.js
// Este archivo configura Express: middlewares, rutas y manejo de errores
require('dotenv').config();
const express = require('express');

const app = express();

// — MIDDLEWARES GLOBALES

// express.json() permite leer el body de las peticiones POST/PUT en formato JSON
// Sin este middleware, req.body siempre sería undefined
app.use(express.json());

// Middleware de logs: muestra en consola cada petición que llega
app.use((req, res, next) => {
  const timestamp = new Date().toISOString().substring(11, 19);
  console.log(`[${timestamp}] ${req.method} ${req.path}`);
  next(); // Pasar al siguiente middleware o ruta
});

// — RUTAS

// Ruta raíz: verifica que el servidor funciona
app.get('/', (req, res) => {
  res.json({
    mensaje: 'StudySync API funcionando',
    version: '1.0.0',
    endpoints: ['/api/sesiones', '/auth/register', '/auth/login', '/api-docs']
  });
});

// Las rutas principales se importan aquí (se agregan en el Paso 4)
// app.use('/api/sesiones', require('./routes/sesiones'));
// app.use('/auth', require('./routes/auth'));

// — MANEJO DE ERRORES GLOBAL

// Este middleware de 4 parámetros SIEMPRE va AL FINAL de todo
// Captura cualquier error que haya ocurrido en las rutas anteriores
app.use((err, req, res, next) => {
  console.error('[ERROR]', err.message);
  res.status(err.status || 500).json({
    error: err.message || 'Error interno del servidor',
    timestamp: new Date().toISOString(),
    ruta: req.path
  });
});
```



```
});

module.exports = app;
```

### 3.2 Crear src/server.js — El punto de entrada

server.js es quien enciende el servidor. En los próximos pasos agregará Socket.io, pero por ahora empieza simple.

```
► src/server.js
// src/server.js
// Punto de entrada principal - inicia el servidor HTTP
require('dotenv').config();
const app = require('./app');

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log('=====');
  console.log(` StudySync API · http://localhost:${PORT}`);
  console.log(` Modo: ${process.env.NODE_ENV || 'development'}`);
  console.log('=====');
});
```

### 3.3 Ejecutar y verificar

```
► INICIAR EL SERVIDOR
# En VS Code: abrir la terminal con Ctrl+` (backtick)
npm run dev
```

```
► RESULTADO ESPERADO

=====
StudySync API · http://localhost:3000
Modo: development
=====
```

Abrir Thunder Client en VS Code (ícono del rayo en la barra lateral) → clic en 'New Request' → seleccionar GET → escribir `http://localhost:3000` → clic en Send.

```
► RESULTADO ESPERADO
{
  "mensaje": "StudySync API funcionando",
  "version": "1.0.0",
  "endpoints": ["/api/sesiones", "/auth/register", "/auth/login", "/api-docs"]
}
```

💡 **nodemon está vigilando los archivos. Cada vez que guardes un cambio (Ctrl+S), el servidor se reinicia automáticamente. Verás el mensaje de la terminal actualizarse solo.**

#### 🐛 Si algo falla en este paso

| Error                        | Solución                                                                                      |
|------------------------------|-----------------------------------------------------------------------------------------------|
| Cannot find module 'express' | El npm install no se ejecutó correctamente. Ejecutar: <code>npm install express dotenv</code> |

|                                                   |                                                                                                        |
|---------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>EADDRINUSE: address already in use :::3000</b> | El puerto 3000 ya está en uso. Cambiar PORT=3001 en .env o cerrar otra terminal que tenga el servidor. |
| <b>nodemon: command not found</b>                 | Ejecutar: npm install -D nodemon. Si persiste: npx nodemon src/server.js                               |

## PASO 4 CRUD de Sesiones — Los 5 Endpoints

Tiempo estimado: 45 minutos · El núcleo de la API

🎯 Los endpoints son como los botones del menú de un cajero automático: cada uno hace una cosa específica. GET es 'consultar saldo', POST es 'depositar dinero', PUT es 'actualizar datos', DELETE es 'cancelar tarjeta'.

### 4.1 Crear el controlador — la lógica de negocio

El controlador contiene la lógica de qué hacer cuando llega cada petición. Está separado de las rutas para que el código sea más fácil de leer y mantener. Crear el archivo `src/controllers/sesionesController.js`:

```

▶ src/controllers/sesionesController.js
// src/controllers/sesionesController.js
// Contiene la lógica de negocio para las sesiones de estudio
// Por ahora usa un arreglo en memoria – el Paso 11 lo migra a Supabase

let sesiones = []; // Base de datos temporal en memoria
let nextId = 1; // Contador autoincrementable de IDs

// — GET /api/sesiones

// Devuelve todas las sesiones disponibles
const listar = async (req, res) => {
  res.json({
    ok: true,
    total: sesiones.length,
    datos: sesiones
  });
};

// — GET /api/sesiones/:id

// Devuelve UNA sesión buscando por su ID
const obtenerUna = async (req, res) => {
  const id = parseInt(req.params.id); // Convertir string a número
  const sesion = sesiones.find(s => s.id === id);

  if (!sesion) {
    // 404 = Not Found: el recurso no existe
    return res.status(404).json({ error: `Sesión ${id} no encontrada` });
  }

  res.json(sesion);
};

// — POST /api/sesiones

// Crea una nueva sesión con los datos del body
const crear = async (req, res) => {
  const { titulo, descripcion, fechaHora, materia } = req.body;

  // Validación: título es obligatorio
  if (!titulo || titulo.trim() === '') {
    // 400 = Bad Request: el cliente envió datos incorrectos
    return res.status(400).json({
      error: 'El campo titulo es obligatorio',
    });
  }
};

```

```

        campos_requeridos: ['titulo'],
        campos_opcionales: ['descripcion', 'fechaHora', 'materia']
    });
}

const nuevaSesion = {
    id: nextId++,
    titulo: titulo.trim(),
    descripcion: descripcion || '',
    materia: materia || 'General',
    fechaHora: fechaHora || new Date().toISOString(),
    completada: false,
    creadaEn: new Date().toISOString()
};

sesiones.push(nuevaSesion);

// 201 = Created: se creó un nuevo recurso exitosamente
res.status(201).json(nuevaSesion);
};

// — PUT /api/sesiones/:id


---


// Actualiza una sesión existente (reemplaza todos sus campos)
const actualizar = async (req, res) => {
    const id = parseInt(req.params.id);
    const indice = sesiones.findIndex(s => s.id === id);

    if (indice === -1) {
        return res.status(404).json({ error: `Sesión ${id} no encontrada` });
    }

    // Conservar el id original y la fecha de creación, actualizar el resto
    sesiones[indice] = {
        ...sesiones[indice], // Campos originales
        ...req.body,         // Nuevos valores del body
        id: id,               // El ID nunca cambia
        actualizadaEn: new Date().toISOString()
    };

    res.json(sesiones[indice]);
};

// — DELETE /api/sesiones/:id


---


// Elimina una sesión del arreglo
const eliminar = async (req, res) => {
    const id = parseInt(req.params.id);
    const longitudAnterior = sesiones.length;

    sesiones = sesiones.filter(s => s.id !== id);

    if (sesiones.length === longitudAnterior) {
        return res.status(404).json({ error: `Sesión ${id} no encontrada` });
    }

    res.json({ ok: true, mensaje: `Sesión ${id} eliminada correctamente` });
};

module.exports = { listar, obtenerUna, crear, actualizar, eliminar };

```

## 4.2 Crear las rutas

Las rutas mapean las URLs con los controladores. Son como el índice de un libro: 'para el capítulo X, ir a la página Y'. Crear `src/routes/sesiones.js`:

```
► src/routes/sesiones.js
// src/routes/sesiones.js
// Define qué función del controlador se ejecuta para cada verbo + ruta
const express = require('express');
const router = express.Router();
const ctrl = require('../controllers/sesionesController');

// GET /api/sesiones → listar todas
// GET /api/sesiones/:id → obtener una
// POST /api/sesiones → crear nueva
// PUT /api/sesiones/:id → actualizar
// DELETE /api/sesiones/:id → eliminar

router.get('/', ctrl.listar);
router.get('/:id', ctrl.obtenerUna);
router.post('/', ctrl.crear);
router.put('/:id', ctrl.actualizar);
router.delete('/:id', ctrl.eliminar);

module.exports = router;
```

## 4.3 Conectar las rutas a app.js

Ahora hay que decirle a Express que use estas rutas. Abrir `src/app.js` y reemplazar el comentario de las rutas:

```
► ACTUALIZAR src/app.js
// En src/app.js – reemplazar esta línea comentada:
// app.use('/api/sesiones', require('./routes/sesiones'));
// app.use('/auth', require('./routes/auth'));

// POR ESTAS LÍNEAS (sin comentarios):
const sesionesRouter = require('./routes/sesiones');
app.use('/api/sesiones', sesionesRouter);
// La ruta /auth se agrega en el Paso 9 (JWT)
```

## 4.4 Probar los 5 endpoints con Thunder Client

Con el servidor corriendo (`npm run dev`), abrir Thunder Client y probar cada endpoint en orden:

| # | Verbo + URL                                 | Body (JSON)                                    | Status esperado | Respuesta esperada         |
|---|---------------------------------------------|------------------------------------------------|-----------------|----------------------------|
| 1 | GET<br>http://localhost:3000/api/sesiones   | (vacío)                                        | 200 OK          | {ok:true,total:0,datos:[]} |
| 2 | POST<br>http://localhost:3000/api/sesiones  | {"titulo":"Redis Pub/Sub","materia":"Prog IV"} | 201 Created     | {id:1,titulo:...}          |
| 3 | GET<br>http://localhost:3000/api/sesiones/1 | (vacío)                                        | 200 OK          | {id:1,titulo:...}          |

|          |                                                |                     |                      |                                       |
|----------|------------------------------------------------|---------------------|----------------------|---------------------------------------|
| <b>4</b> | PUT<br>http://localhost:3000/api/sesiones/1    | {"completada":true} | 200 OK               | {id:1,completada:tru<br>e}            |
| <b>5</b> | GET<br>http://localhost:3000/api/sesiones/999  | (vacío)             | <b>404 Not Found</b> | "error":"Sesión 999<br>no encontrada" |
| <b>6</b> | DELETE<br>http://localhost:3000/api/sesiones/1 | (vacío)             | 200 OK               | "ok":true,"mensaje"..<br>.."          |

 **Para configurar el body JSON en Thunder Client: clic en la pestaña 'Body' → seleccionar 'JSON' → escribir el objeto JSON. Thunder Client agrega automáticamente el header Content-Type: application/json.**

 **Al finalizar el Paso 4 debes tener: 5 endpoints funcionando. GET devuelve 200, POST devuelve 201, IDs inexistentes devuelven 404, campos faltantes devuelven 400.**

## PASO 5 Deploy en Render — La API en Internet

Tiempo estimado: 20 minutos · Actividad 1 completada

🎯 Hasta ahora tu API vive solo en tu computadora. Deploying es como mudarte de un apartamento privado a un local comercial con dirección pública: cualquier persona en el mundo puede visitarte.

### 5.1 Hacer push del código actualizado

#### ▶ COMMIT Y PUSH

```
git add .
git commit -m "feat(api): CRUD completo de sesiones con 5 endpoints"
git push origin main
```

### 5.2 Crear el Web Service en Render

1. Ir a [render.com](https://render.com) → hacer clic en 'New +' → seleccionar 'Web Service'.
2. Clic en 'Connect account' → seleccionar GitHub → autorizar Render → buscar el repositorio `studysync-api` → clic en Connect.
3. Configurar el servicio: Name = `studysync-api`, Region = Oregon (US West), Branch = `main`.
4. Build Command = `npm install` (Render lo detecta automáticamente).
5. Start Command = `npm start` (usa el script definido en `package.json`).
6. Ir a la sección 'Environment' → clic en 'Add Environment Variable' → agregar `PORT=3000`.
7. Clic en 'Create Web Service' → esperar 3-5 minutos mientras se construye.

#### ▶ RESULTADO ESPERADO

Your service is live at: <https://studysync-api.onrender.com>

### 5.3 Verificar en producción

Con Thunder Client, cambiar la URL base a la URL de Render y probar el endpoint raíz:

#### ▶ PROBAR EN PRODUCCIÓN

```
GET https://studysync-api.onrender.com/
```

#### ▶ RESULTADO ESPERADO

```
{"mensaje":"StudySync API funcionando","version":"1.0.0",...}
```

💡 Render desplegará automáticamente cada vez que hagas `git push` a la rama `main`. Esto se llama CD (Continuous Deployment) y ya lo tienes configurado gratis.

### 🐛 Si el deploy falla en Render

| Error en logs de Render             | Solución                                                                                                                                                                        |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 'npm start' failed — missing script | Verificar que <code>package.json</code> tiene: <code>scripts.start = 'node src/server.js'</code>                                                                                |
| Cannot find module 'express'        | El <code>node_modules</code> no se instaló. Verificar que <code>package.json</code> tiene <code>express</code> en <code>dependencies</code> (no <code>devDependencies</code> ). |
| Port 10000 already in use           | Render asigna su propio puerto. Verificar que <code>app.listen</code> usa <code>process.env.PORT    3000</code> (nunca puerto fijo).                                            |





## PASO 6 Configurar Upstash Redis

Tiempo estimado: 15 minutos · La infraestructura de mensajería

🎯 **Redis es como la memoria RAM del sistema: ultra-rápida pero temporal. Upstash es una empresa que te da una porción de esa RAM en la nube, gratis. La usaremos como el canal de radio por donde se transmiten los eventos en tiempo real.**

### 6.1 Crear la base de datos en Upstash

8. Ir a [upstash.com](https://upstash.com) → clic en 'Start for Free' → Login with GitHub (más rápido que crear cuenta nueva).
9. En el dashboard → clic en 'Create Database'.
10. Name: studysync-progiv. Type: Redis. Region: US-East-1. Clic en Create.
11. Esperar 5 segundos → la BD aparece en el dashboard.
12. Clic en la BD creada → ir a la pestaña 'Details' → encontrar el campo 'REDIS\_URL' → clic en el ícono de copiar.

La REDIS\_URL se ve así (con tus valores reales):

```
► FORMATO DE REDIS_URL
redis://default:TuContraseñaLarga@tu-host.upstash.io:6379
```

### 6.2 Guardar en .env

Abrir el archivo .env y reemplazar el valor PENDIENTE de REDIS\_URL:

```
► ACTUALIZAR .env
# Reemplazar esta línea:
REDIS_URL=redis://default:PENDIENTE@PENDIENTE:6379

# Con la URL real copiada de Upstash (ejemplo):
REDIS_URL=redis://default:abc123xyz@us1-cute-lion-12345.upstash.io:6379
```

🚫 **Después de actualizar .env, NUNCA hacer git add .env ni git add . sin verificar que .env está en .gitignore. Si subes las credenciales a GitHub, cualquiera puede usar tu Redis gratis a tu nombre.**

### 6.3 Verificar la conexión desde la consola web de Upstash

En el dashboard de Upstash → clic en tu BD → pestaña 'CLI' → escribir:

```
► PROBAR DESDE CONSOLA WEB DE UPSTASH
PING
```

```
► RESULTADO ESPERADO
PONG
```

Si responde PONG, la conexión funciona. También puedes probar:

```
► COMANDOS BÁSICOS
SET prueba "Hola desde StudySync"
GET prueba
DEL prueba
```

## 6.4 Instalar ioredis en el proyecto

---

► **INSTALAR CLIENTE REDIS**

```
npm install ioredis
```

► **RESULTADO ESPERADO**

```
added 8 packages, and audited 66 packages in 2s
```

✓ **Al finalizar el Paso 6: Upstash BD creada. REDIS\_URL en .env actualizada. ioredis instalado. PING responde PONG desde la consola web.**

## PASO 7 El Cliente Redis y el Publicador

Tiempo estimado: 30 minutos · El locutor de la radio

🎧 **Redis necesita dos conexiones separadas: una para publicar (el locutor) y otra para escuchar (los oyentes). Es como en una radio: el locutor usa un micrófono diferente al altavoz. Nunca pueden compartir el mismo cable.**

### 7.1 Crear el cliente Redis compartido

Crear el archivo `src/redis/client.js` que exporta las dos conexiones:

```
▶ src/redis/client.js
// src/redis/client.js
// Exporta DOS conexiones Redis independientes:
// - pub: para publicar mensajes (no puede suscribirse mientras publica)
// - sub: para suscribirse a canales (no puede publicar mientras escucha)
// Esta separación es OBLIGATORIA en Redis Pub/Sub
require('dotenv').config();
const Redis = require('ioredis');

const REDIS_URL = process.env.REDIS_URL;

if (!REDIS_URL || REDIS_URL.includes('PENDIENTE')) {
  console.error('❌ REDIS_URL no configurada en .env');
  process.exit(1); // Detener el servidor si Redis no está configurado
}

// Opciones compartidas de conexión
const opciones = {
  maxRetriesPerRequest: 3, // Reintentar 3 veces si falla
  retryStrategy: (times) => {
    if (times > 3) return null; // Después de 3 intentos, rendirse
    return Math.min(times * 200, 1000); // Esperar 200ms, 400ms, 600ms...
  }
};

// Conexión para PUBLICAR eventos
const pub = new Redis(REDIS_URL, opciones);
pub.on('connect', () => console.log('✓ Redis Pub: conectado a Upstash'));
pub.on('error', (e) => console.error('❌ Redis Pub error:', e.message));

// Conexión para SUSCRIBIRSE a canales
const sub = new Redis(REDIS_URL, opciones);
sub.on('connect', () => console.log('✓ Redis Sub: conectado a Upstash'));
sub.on('error', (e) => console.error('❌ Redis Sub error:', e.message));

module.exports = { pub, sub };
```

### 7.2 Integrar el publicador en el controlador de sesiones

Cada vez que se crea una sesión nueva, publicar un evento en el canal Redis. Abrir `src/controllers/sesionesController.js` y modificarlo:

```
▶ MODIFICAR src/controllers/sesionesController.js
// src/controllers/sesionesController.js
// AGREGAR al inicio del archivo (después de la primera línea):
const { pub } = require('../redis/client');
```

```
// ... (el resto del código queda igual) ...

// MODIFICAR la función 'crear' – agregar el publish DESPUÉS de push():
const crear = async (req, res) => {
  const { titulo, descripcion, fechaHora, materia } = req.body;

  if (!titulo || titulo.trim() === '') {
    return res.status(400).json({ error: 'El campo titulo es obligatorio' });
  }

  const nuevaSesion = {
    id: nextId++,
    titulo: titulo.trim(),
    descripcion: descripcion || '',
    materia: materia || 'General',
    fechaHora: fechaHora || new Date().toISOString(),
    completada: false,
    creadaEn: new Date().toISOString()
  };

  sesiones.push(nuevaSesion);

  // ✨ NUEVO: publicar evento en Redis DESPUÉS de guardar la sesión
  // pub.publish(canal, mensaje_como_string_json)
  await pub.publish('study:sesion:creada', JSON.stringify({
    tipo: 'sesion:creada',
    payload: nuevaSesion,
    timestamp: new Date().toISOString()
  }));
  console.log('[Redis] Evento publicado: sesion:creada →', nuevaSesion.titulo);

  res.status(201).json(nuevaSesion);
};
```

⚠ El `await pub.publish()` es rápido (~1-2ms con Upstash). No hace lento el endpoint porque Upstash está optimizado para baja latencia. Si Redis falla, el `try/catch` del middleware global captura el error sin tirar el servidor.

## PASO 8 El Suscriptor y Socket.io — Tiempo Real

Tiempo estimado: 1 hora · El momento WOW del sistema

🔗 **Socket.io es como una línea telefónica permanente entre el servidor y el navegador. Mientras HTTP es como enviar cartas (petición → respuesta → conexión cerrada), Socket.io es como una llamada de WhatsApp que siempre está activa. El servidor puede hablar en cualquier momento sin que el navegador pregunte.**

### 8.1 Instalar Socket.io

#### ► INSTALAR SOCKET.IO

```
npm install socket.io @socket.io/redis-adapter
```

### 8.2 Crear el suscriptor de notificaciones

El suscriptor escucha los eventos Redis y los retransmite al navegador via Socket.io. Crear src/subscribers/notificaciones.js:

```
► src/subscribers/notificaciones.js
// src/subscribers/notificaciones.js
// Este módulo escucha canales Redis y retransmite los eventos
// a los navegadores conectados mediante Socket.io
const { sub } = require('../redis/client');

// Esta función recibe el objeto 'io' de Socket.io y lo usa para emitir
// Se llama desde server.js después de inicializar Socket.io
const iniciarSuscripciones = (io) => {
  // Suscribirse a TODOS los canales que empiecen con 'study:'
  // El asterisco (*) es un wildcard (comodín)
  sub.psubscribe('study:*', (err, count) => {
    if (err) {
      console.error('[Sub] Error al suscribirse:', err.message);
      return;
    }
    console.log(`[Sub] ✓ Escuchando ${count} patrón(es) en Redis...`);
  });

  // Este listener se ejecuta cada vez que llega un mensaje
  // pattern = patrón que coincidió ('study:*')
  // channel = canal exacto ('study:sesion:creada')
  // message = el JSON string que publicó el publicador
  sub.on('pmessage', (pattern, channel, message) => {
    try {
      const evento = JSON.parse(message);
      console.log(`[Sub] Recibido en ${channel}:`, evento.tipo);

      // Emitir el evento a TODOS los navegadores conectados via Socket.io
      // El nombre 'nuevo-evento' es el que escucha el cliente HTML
      io.emit('nuevo-evento', {
        canal: channel,
        ...evento
      });
    } catch (error) {
      console.error('[Sub] Error procesando mensaje:', error.message);
    }
  });
};
```

```
};

module.exports = { iniciarSuscripciones };
```

## 8.3 Actualizar server.js para agregar Socket.io

Reemplazar el contenido de src/server.js completo:

```
► src/server.js ACTUALIZADO
// src/server.js – Versión final con Socket.io + Redis
require('dotenv').config();
const http = require('http'); // Módulo HTTP nativo de Node.js
const { Server } = require('socket.io'); // Motor de WebSockets
const app = require('./app'); // Express configurado
const { pub, sub } = require('./redis/client'); // Conexiones Redis

// 1. Crear servidor HTTP sobre Express
// (Socket.io necesita el servidor HTTP, no Express directamente)
const servidor = http.createServer(app);

// 2. Crear instancia de Socket.io sobre el servidor HTTP
const io = new Server(servidor, {
  cors: { origin: '*' }, // En producción, restringir al dominio del frontend
  transports: ['polling', 'websocket'] // polling primero para compatibilidad con Render free
});

// 3. Iniciar las suscripciones Redis (pasa 'io' para poder emitir)
const { iniciarSuscripciones } = require('./subscribers/notificaciones');
iniciarSuscripciones(io);

// 4. Manejar conexiones WebSocket de navegadores
io.on('connection', (socket) => {
  console.log(`[WS] Cliente conectado: ${socket.id}`);

  socket.on('disconnect', () => {
    console.log(`[WS] Cliente desconectado: ${socket.id}`);
  });
});

// 5. Iniciar el servidor (servidor, no app – para que Socket.io funcione)
const PORT = process.env.PORT || 3000;
servidor.listen(PORT, () => {
  console.log('=====');
  console.log(` StudySync API + WebSocket · http://localhost:${PORT}`);
  console.log(`/api/sesiones /auth /api-docs`);
  console.log('=====');
});
```

## 8.4 Crear la interfaz web en tiempo real

Crear public/index.html — la página que se actualiza sola sin recargar:

```
► public/index.html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>StudySync – Notificaciones en Tiempo Real</title>
  <!-- Socket.io Client: auto-servido por el servidor -->
```

```

<script src="/socket.io/socket.io.js"></script>
<style>
  body { font-family: Arial, sans-serif; max-width: 700px; margin: 40px auto; padding:
20px; }
  h1 { color: #1A56DB; }
  #status { padding: 8px; border-radius: 4px; margin-bottom: 16px; }
  .conectado { background: #D1FAE5; color: #065F46; }
  .desconectado { background: #FEE2E2; color: #7F1D1D; }
  #eventos { list-style: none; padding: 0; }
  #eventos li {
    padding: 12px; margin: 8px 0; border-radius: 8px;
    background: #FFF6FF; border-left: 4px solid #1A56DB;
    animation: aparecer 0.3s ease;
  }
  @keyframes aparecer { from { opacity:0; transform:translateY(-8px); } to { opacity:1; } }
  .hora { font-size: 11px; color: #64748B; }
</style>
</head>
<body>
  <h1>🦋 StudySync — Eventos en Tiempo Real</h1>
  <div id="status" class="desconectado">● Conectando...</div>
  <ul id="eventos">
    <li style="color:#64748B">Esperando eventos... (crea una sesión con Thunder Client)</li>
  </ul>

  <script>
    // Conectar al servidor Socket.io automáticamente
    const socket = io();

    socket.on('connect', () => {
      document.getElementById('status').textContent = '● Conectado al servidor en tiempo
real';
      document.getElementById('status').className = 'conectado';
    });

    socket.on('disconnect', () => {
      document.getElementById('status').textContent = '● Desconectado';
      document.getElementById('status').className = 'desconectado';
    });

    // Escuchar el evento 'nuevo-evento' que emite el suscriptor Redis
    socket.on('nuevo-evento', (evento) => {
      const lista = document.getElementById('eventos');
      const item = document.createElement('li');
      const hora = new Date(evento.timestamp).toLocaleTimeString();

      item.innerHTML = `
        <strong>${evento.canal}</strong>: ${evento.payload?.titulo ||
JSON.stringify(evento.payload)}
        <br><span class="hora">🕒 ${hora} · Socket ID: ${socket.id}</span>
      `;

      // Mostrar el más reciente primero
      lista.insertBefore(item, lista.firstChild);
    });
  </script>
</body>
</html>

```

## 8.5 Servir el HTML desde Express

Agregar en `src/app.js` (después de los middlewares, antes de las rutas):

```
► AGREGAR EN src/app.js
// En src/app.js – agregar esta línea para servir archivos estáticos
// express.static('public') sirve todos los archivos de la carpeta public/
app.use(express.static('public'));
// Ahora http://localhost:3000 muestra public/index.html
```

## 8.6 Probar el sistema en tiempo real — el momento WOW

Este es el momento más impactante del sistema. Seguir estos pasos en orden:

13. Terminal: asegurarse de que el servidor está corriendo con `npm run dev`.
14. Abrir DOS pestañas del navegador en `http://localhost:3000`. Ambas deben mostrar '● Conectado'.
15. En Thunder Client: hacer POST `http://localhost:3000/api/sesiones` con body: `{"titulo":"Sesión Redis en tiempo real", "materia":"Prog IV"}`.
16. Observar: el evento aparece simultáneamente en AMBAS pestañas sin recargar la página.

```
► RESULTADO ESPERADO
[Redis] Evento publicado: sesion:creada → Sesión Redis en tiempo real
[Sub] Recibido en study:sesion:creada: sesion:creada
[WS] Enviado a 2 cliente(s)
```

💡 Si abris 5 pestañas del navegador, los 5 ven el evento al mismo tiempo. Eso es la arquitectura Pub/Sub: el publicador no sabe cuántos suscriptores hay, y los suscriptores no saben de dónde vienen los mensajes.

### 🐛 Si algo falla en el Paso 8

| Error                                           | Solución                                                                                                                                                                        |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CERTIFICATE_VERIFY_FAILED (solo Python)</b>  | No aplica en Node.js. En Python: <code>redis.from_url(url, ssl_cert_reqs=None)</code>                                                                                           |
| <b>Sub no recibe mensajes</b>                   | Verificar que pub y sub vienen de <code>client.js</code> (son dos objetos diferentes, no el mismo).                                                                             |
| <b>UI no muestra el evento</b>                  | Verificar que <code>app.use(express.static('public'))</code> está en <code>app.js</code> . Verificar nombre del archivo: <code>index.html</code> (no <code>Index.html</code> ). |
| <b>Socket.io 404 en /socket.io/socket.io.js</b> | El servidor HTTP debe crearse con <code>http.createServer(app)</code> , no con <code>app.listen()</code> . Verificar <code>server.js</code> .                                   |

✅ Al finalizar el Paso 8: eventos Redis viajan de Thunder Client → API → Redis → suscriptor → Socket.io → 2 pestañas del navegador en tiempo real. Si esto funciona, tienes el componente más complejo del sistema.